

Projeto Prático #2: O projeto deve ser desenvolvido por grupos de *no máximo* três alunos. É estritamente proibida a partilha de código entre alunos de grupos diferentes.

Plágio: Além de inspeção manual, será também executado um software de deteção de plágio no código-fonte do projeto. *Todos* os alunos que submeterem código obtido através de plágio terão os seus projetos anulados.

Uso de Inteligência Artificial: É permitido utilizar ferramentas de IA como o ChatGPT, Copilot, DeepSeek e similares. No entanto, todos os membros do grupo devem ser capazes de compreender e *explicar* o projeto *na sua totalidade* aos docentes quando pedido. Se houver código no projeto que o grupo não consiga explicar, o projeto será considerado como *plágio*.

Submissão: A data de submissão do projeto é **03/05/2026** às 23:59. A descrição do processo de submissão pode ser encontrada no enunciado.

TascaEats: Delivery de Comida

1 Contextualização

Ao perceber o avanço do projeto na entrega anterior, a a consultoria espanhola **Compadres Sempre Saciados (CSS¹)** percebeu que havia muitos pontos que não tinham ficado claros e que poderiam ser melhorados. E concomitantemente, será necessário também desenvolver duas interfaces gráficas (uma web e uma nativa) para a aplicação.

Dos pontos que não haviam ficados claros com o cliente, destacam-se os seguintes:

- Deve-se permitir ao cliente avaliar o restaurante depois de realizar uma compra.
- Um mesmo pedido pode conter produtos de diferentes restaurantes.
- Um cliente pode ter várias moradas associadas, e pode escolher uma delas no momento de fazer um pedido ou utilizar uma morada nova.
- O sistema deve poder associar, automaticamente, um entregador a um pedido.
- O sistema permitir a distinção dos diferentes tipos de pagamento pelas suas características (e.g., para pagamento com Multibanco, guardar a bandeira do cartão utilizada; para MBWay, permitir associar um número de telemóvel; para Numerário, guardar informação sobre o troco, ...). Não é necessário implementar a lógica de processamento de pagamentos.
- Deve ser permitido a partilha de um mesmo menu entre diversos restaurantes (e.g., uma cadeia de restaurantes pode ter um menu comum). O sistema deve permitir a gestão de menus e a associação desses menus a restaurantes.

¹Entidade fictícia

As duas interfaces gráficas devem ser desenvolvidas utilizando Thymeleaf para a versão web e JavaFX para a versão nativa. Ambas as interfaces devem permitir a interação com casos de uso específicos e devem ser implementadas de forma a garantir uma experiência de utilizador consistente e intuitiva. A interface web deve comunicar-se com o backend através de endpoints REST, enquanto a interface nativa deve utilizar gRPC para comunicação com o backend.

Nesta **Fase 2**, o foco será na revisão do *backend* e no desenvolvimento do *frontend*.

2 Casos de Uso

Os casos de uso foram revistos e atualizados com base nas novas informações obtidas do cliente, e agora incluem também os filtros necessários para realizar as buscas. Os casos de uso que estiverem em **azul** devem ser implementados na interface web; os que estiverem em **roxo**, na interface nativa; os que estiverem em **castanho** em ambas as interfaces.

- A. Login com autenticação (Nota: vamos fazer mock. Qualquer palavra-passe será aceite con-
tanto que o utilizador seja válido)².
- B. Registro de Clientes.
- C. Registro de Administradores e Entregadores.
- D. Verificar, remover e atualizar utilizadores.
- E. Buscar por restaurantes (filtro definido pelo utilizador).
- F. Gerir Restaurantes.
- G. Gerir Menus.
- H. Gerir Produtos.
- I. Buscar produtos de um restaurante (filtro definido pelo utilizador).
- J. Criação de um pedido.
- K. Registo e processamento do pagamento associado a um pedido.
- L. Atualização do estado do pedido.
- M. Atribuição de um entregador a um pedido
- N. Cancelamento de um pedido

3 Requisitos Não Funcionais

O projecto tem ainda os seguintes requisitos:

- Os requisitos não funcionais da Fase 2 são cumulativos, ou seja, os casos de uso implementados na Fase 1 devem continuar a funcionar corretamente, e os novos casos de uso devem ser implementados de forma a garantir a integração com os casos de uso anteriores.

²O sistema deve ser capaz de distinguir os diferentes tipos de utilizadores (clientes, entregadores e administradores).

- **Submissão de vídeos demonstrativos da execução dos casos de uso**
- O projecto correrá sempre num ambiente Docker.
- O controlo da participação de cada membro do grupo será feito via atividade no repositório git, com base no número e qualidade dos *commits*.
- A interface gráfica implementada de duas maneiras, cada uma delas deverá realizar uma parte dos casos de uso. Uma das interfaces será através do browser numa interface web usando Thymeleaf e a outra uma interface nativa com JavaFX.
- O repositório deve aceitar apenas código com o nível de qualidade aceite pela equipa (ex.: *pre-commit*, testes).
- O projecto correrá num ambiente Docker.
- O controlo da participação de cada membro do grupo será feito via atividade no repositório git, com base no número e qualidade dos *commits*.

4 Tarefas da Fase 2

As tarefas da fase 2 presumem que as tarefas da fase 1 foram realizadas com sucesso. Caso alguma das tarefas ainda esteja pendente, é necessária a sua conclusão antes de iniciar a fase 2.

O cliente notou que o sistema de buscas era muito simples e portanto pediu que fossem adicionados novos casos de uso para permitir buscas mais complexas através do uso de filtros. Os filtros que devem ser implementados são os seguintes:

- Filtros do Utilizador:
 - Filtro por nome.
 - Filtro por tipo de utilizador.
 - Filtro por número de pedidos realizados (apenas para clientes).
 - Filtro por número de entregas realizadas (apenas para entregadores).
- Filtros do Restaurante:
 - Filtro por nome.
 - Filtro por número de pedidos realizados.
 - Filtro por número de avaliações recebidas.
 - Filtro pela morada.
 - Filtro por tipo de cozinha (e.g., italiana, chinesa, ...).
 - Filtro por horário de funcionamento (e.g., aberto, fechado, ...).
 - Filtro por preço médio dos pratos.
- Filtros da Produtos:
 - Filtro por nome.
 - Filtro por preço.

- Filtro por categoria (e.g., entrada, prato principal, sobremesa, ...).
- Filtro por disponibilidade (e.g., disponível, esgotado, ...) (apenas administradores e entregadores).
- Por popularidade (número de vezes que o produto foi pedido, dentro de um intervalo de tempo).
- Filtros do Menu:
 - Filtro por nome.
 - Filtro por número de produtos associados.
 - Filtro por preço médio dos produtos associados.

Pretende-se implementar a interface gráfica que permita aos utilizadores realizarem os casos de uso de forma interativa. A interface gráfica será desenvolvida de duas formas diferentes: i) através do browser, numa interface web, com renderização server-side onde será possível realizar as consultas a informação do sistema (casos de uso marcados a azul) e através de uma aplicação nativa desenvolvida em JavaFX, onde será possível inserir dados ao sistema (casos de uso marcados a roxo). Os casos de uso em castanho devem ser implementadas nas duas interfaces. De forma a demonstrar a funcionalidade do sistema, devem fazer um vídeo para cada interface gráfica a mostrar o funcionamento do sistema (recomendações sobre os vídeos estão disponíveis no Moodle).

4.1 Validação

Assim como na primeira fase, devem garantir que os casos de uso estão implementados e demonstrados nos vídeos. Garantam também que as mudanças solicitadas pelo cliente estão implementadas e demonstradas. Por exemplo:

- Ao solicitar um pedido, pode não ser necessário associar um entregador (associado automaticamente). O entregador deve ser válido.
- Uma modificação num item de um menu partilhado deve refletir-se em todos os restaurantes que partilham esse menu.
- Deve ser permitido entregar em qualquer uma das moradas do cliente (ou ele pode inserir uma nova morada no momento de fazer o pedido).

Este não é um conjunto exaustivo das regras de negócio, serve somente para ilustrar casos que o sistema deve garantir e isso deve ser demonstrado nos vídeos.

Para validar o funcionamento da aplicação, também devem garantir que o modelo de base de dados gerado deve ser capaz de responder a questões de negócio como estas:

- No caso de pagamento com numerário, qual é a média do troco?
- Qual é o item mais pedido de um restaurante?
- Qual o entregador com mais entregas para um restaurante específico?
- Qual o restaurante mais popular de uma franquia específica?
- Qual é o cliente que mais pedidos fez num intervalo de tempo específico?

5 Como Entregar

Para entregar o trabalho, basta criar uma tag chamada *fase2* e enviá-la para o repositório. O repositório deverá conter o código-fonte do projecto, o ficheiro `docker-compose.yml` necessário para correr a aplicação e a base de dados. Não se esqueça de incluir os links dos vídeos no ficheiro `README.md`.

```
git tag fase2
git push origin fase2
```

Deve-se confirmar que o vosso projecto está acessível à conta CSS000 na tag *fase2*. Caso contrário, terão 0 nesta entrega. É importante garantir que o ambiente de execução possa ser reproduzido pela equipa docente sem erros de compilação. Para tal, recomenda-se executar o projecto em cada um dos computadores dos membros da equipa. O projecto **obrigatoriamente** deverá correr num ambiente Docker. Falhas em **EXECUTAR** o projecto, num ambiente Docker, pela equipa docente, resultarão em **grave penalização nesta entrega**. Casos de uso que não sejam apresentados ou que apresentem falhas de execução, serão considerados como **não implementados**. Casos de uso parcialmente apresentados terão uma nota também parcial.

6 Critérios de Avaliação

Esta entrega corresponde a 35% da nota do projecto. Os restantes 65% serão avaliados na primeira fase e terceira fase. Para perceberem onde se devem focar, segue uma lista de critérios que serão avaliados:

6.1 Processo de Desenvolvimento e Demonstração

- Granularidade e frequência dos *commits*. Todos os membros do grupo devem contribuir de forma equitativa. *Disparidades significativas serão penalizadas*.
- Qualidade das mensagens de *commit*.
- Clareza na demonstração, funcionamento real da API REST apresentada e capacidade de explicação do trabalho desenvolvido pelo grupo nos vídeos.
- Quantidade e qualidade das regras de negócio implementadas e demonstradas nos vídeos.
- Quão bem os vários padrões leccionados são aplicados no projecto

6.2 Requisitos

- Em que medida os requisitos funcionais e não funcionais são abrangidos pela modelação.
- Usabilidade da interface desenhada para cumprir os objectivos.

Nota Final: Em caso de dúvidas, utilizem o fórum de discussão no Moodle. Boa trabalho com o desenvolvimento do *TascaEats*!